

תיאור מקיף: מחזור החיים של שלב העברת הנתונים (שלב 2)

בחלק זה של הדיון שלנו ב-TCP, נתאר את "הקרביים" של מנגנון העברת הנתונים של TCP. את שלב מס' 2.

שלב ה-data exchange.

שלב זה הוא הלב הפועם של העברת הנתונים האמינה.

אנו מתמקדים בתסריט שבו לקוח (Chrome) שולח הודעת GET של 2000 בתים לשרת (Apache). התיאור מציג את הדינמיקה בין מרחב המשתמש (האפליקציה) לבין מנוע ה-TCP בקרנל (מרחב הליבה).

נריץ "סרט אנימציה" בהילוך איטי (Slow Motion) מרגע שהלקוח לוחץ על ההדק, ועד שהמידע מגיע בבטחה ליעדו.

הנחות:

לצורך הדיוק המתמטי של האנימציה, נשתמש בהנחות הבאות:

MSS = 1460 -

- ה- ISN של הלקוח (Chrome) היה 1,000,000. נקבע בלשב 1. שלב לחיצת היד והקמת הקישור. לכן, כפי שראינו, חבילת ה-SYN לקחה לוגית את מספר 1,000,000, והבית הראשון של נתונים שיישלח כעת יקבל את המספר 1,000,001.

תת שלב מס' 1: היצרן (Chrome) מזרים נתונים אל הקרנל

- **הפעולה:** תהליכון [= Chrome-nthread] במרחב המשתמש [= user space] סיים לבנות את מחרוזת [= string] ה-GET (בגודל 2000 בתים) בזיכרון הפרטי שלו.
וכעת, בתור **היצרן (Producer)**, הוא מבצע קריאת מערכת [= system call]:
send(fd, buffer, 2000, 0)
- **מעבר לליבה:** המעבד 'קופץ' למרחב הליבה [= kernel space].
- מנוע ה-TCP מקבל את 2000 הבתים ומעתיק אותם אל תוך **Buffer השליחה** (sk_sndbuf) של הסוקט, שגודלו 128KB (ולכן יש בו שפע של מקום).
- **חזרת שליטה:** מרגע שההעתקה ל-Buffer הליבה הושלמה, פונקציית ה-send() חוזרת ל-Chrome, והתהליכון יכול להמשיך בענייניו.
- ה-Chrome מבחינתו "סיים לשלוח", אבל בפועל – **המידע אפילו לא יצא מכרטיס הרשת!**
שלא לדבר על כך שהמידע כלל לא הגיע ליעד שלו [= ל- http server שב- Apache]
מעכשיו, הקרנל הוא הריבון.

והאינטראקציה לצורך 'מסירת' הודעת ה-GET, ה-2000 בתים, ל-http server, תתבצע בין TCP במחשב הלקוח לבין TCP במחשב השרת.

מעבר לליבה: המעבד עובר מ-Ring 3 (משתמש) ל-Ring 0 (ליבה). הקרנל מעתיק את 2000 הבתים מהזיכרון הפרטי של הדפדפן אל תוך חוצץ השליחה של הסוקט (sk_sndbuf), המוקצה בזיכרון הליבה.

הסדרת זיכרון:

אם ה-sk_sndbuf מלא, אזי ה-send() יחסום את ה-Chrome (התהליכון יעבור למצב "המתנה"). אם יש מקום, הנתונים מועתקים, הפונקציה חוזרת ל-Chrome, והתהליכון "משוחררת" להמשיך, בעוד הקרנל לוקח פיקוד על המשך ההעברה.

תת שלב מס' 2: חיתוך וקיטוע (Segmentation) בקרנל, וניהול הרצף

מנוע ה-TCP מטפל ב-2000 הבתים ששמורים ב-sk_sndbuf. הוא 'יודע' שגודל ה-MSS (Maximum Segment Size) מול השרת הוא 1460 בתים. פרט זה הוסכם אף הוא מול TCP בשרת בשלב לחיצת היד. שלב הקמת הקישור. כפי שהוסכם על ערכו של ה-ISBN.

כפי שראינו בהרצאות הראשונות בקורס, 'חוקי הפיזיקה של הרשת' לא מאפשרים לשלוח 2000 בתים במכה אחת (זה יחרוג מגודל מסגרת ה-Ethernet). מהגודל המקסימלי שה-NIC תוכנן ומומש להעביר.

לכן, הקרנל "חותך" [= מבצע segmentation] את הנתונים לשני אובייקטים (שני sk_buff):

• סגמנט DATA מס' 1:

- **Payload (נתונים):** חותך את 1460 הבתים הראשונים מתוך ה-GET.
- **Sequence Number:** יקבל את המספר 1,000,001.
- **Checksum:** הקרנל מריץ חישוב מתמטי על 1460 הבתים + הכותרת, ושולח את התוצאה כאן (כדי לממש את התחייבות ה-No Error).

רשתות תקשורת, תשפ"ו
TCP Data Exchange
עמוד 2

• סגמנט DATA מס' 2 :

- **Payload (נתונים):** חותך את 540 הבתים שנותרו ($2000 = 540 + 1460$).
- **Sequence Number:** יקבל את המספר **1,001,461** (היינו, $1,000,001 + 1460$). זהו הקסם של זרם הבתים – המספר הסידורי איננו "מספר החבילה", אלא האינדקס של הבית הראשון שנמצא בתוכה.
- **Checksum:** מחושב ספציפית עבור הסגמנט הזה.

נשים לב :

- סגמנט מס' 1 : (1460 בתים), מספר רצף 1,000,001.
- סגמנט מס' 2 : (540 בתים), מספר רצף 1,001,461.

כל סגמנט מקבל Checksum מחושב.
משמש 'להבטיח' no-error.

- בתוך ה-TCB (Transmission Control Block) של הקרנל, מנוע ה-TCP מנהל את הנתונים באמצעות שני מצביעים לוגיים (Sequence Pointers) :
- **SND.UNA:** המצביע על הבית הישן ביותר שטרם אושר.
 - **SND.NXT:** המצביע על הבית הבא שישודר.

תת שלב מס' 3: בקרת עומסים ובקרת זרימה (Congestion Control & Flow)

לפני ש-TCP יורה את הסגמנטים החוצה [= מעביר 'כלפי' היעד, אל TCP במחשב השרת], הוא חייב לוודא שמותר לו

מה פירוש?

כדי לא לחנוק את השרת או את האינטרנט). הוא בודק שני פרמטרים מתוך רשומת ה-tcp_sock :

1. **חלון השליחה / חלון הקבלה של השרת (rwnd):** השרת הצהיר ב-SYN-ACK שיש לו 65,535 בתים פנויים ב-Buffer.
 2. זהו מנגנון ה-**Flow Control** [= בקרת זרימה]. שנועד להגן על 'הנמען' מפני הצפה.
 3. **חלון העומס (cwnd):** TCP שלנו התחיל את החיבור עם חלון התחלתי בגודל 10 MSS (כלומר 14,600 בתים).
- זהו מנגנון ה-**Congestion Control** (בקרת העומסים). שנועד להגן על העומס ברשת.

או יותר מדויק: על הנתבים וה- switches שלאורך המסלול [route =] שבין מחשב הלקוח לבין מחשב השרת.

- **ההחלטה:** הקרנל לוקח את המינימום מבין שניהם (14,600 בתים). מכיוון שסך כל הנתונים שאנו רוצים לשלוח כעת הוא רק 2000 בתים, הקרנל מאשר שידור **מיידי ורציף של שני הסגמנטים יחד**.
אין צורך לשלוח את סגמנט 1 ולהמתין ל-ACK שלו כדי לשלוח את סגמנט 2!
- **פעולת השידור והמשך 'האגירה' ב- buffer השליחה:** שני הסגמנטים נשלחים אל כרטיס הרשת (NIC) ומשודרים לאינטרנט. **אבל – וזה קריטי ל-RDT – העותקים שלהם לא נמחקים מ-Buffer השליחה (sk_sndbuf)!**
- הם נשארים שם "כבני ערובה" למקרה שילכו לאיבוד בדרך (No Loss), ומופעל עליהם שעון עצר (Retransmission Timer).

הערות חשובות:

- הקרנל מנהל חלון שליחה [Send Window =]
עבור כל קישור TCP
- הוא מנהל אותו מרגע הקמת הקישור [= סיום שלב 1] ועד רגע סגירת הקישור [= שלב 3]
- כלומר הוא מנהל אותו לאורך כל שלב 2

- זהו חלון לוגי
- והוא מוגדר 'בתוך' ה- send buffer. על ידי מצביעים.

- גודלו כאמור מוגדר להיות:

$$\text{Send Window} = \min(\text{rwnd}, \text{cwnd})$$

- מכיוון שערכם של rwnd ושל cwnd משתנה/מתעדכן עם הזמן [= לאורך 'חיי הקישור', הרי שעולה מכך כי ערכו של Send Window אף הוא דינמי ומשתנה עם הזמן.
- הוא איננו סטטי.
- הוא יכול לגדול 'להתרחב' ולקטון 'להתכווץ'.

- בספרות מופיע לעיתים קרובות גם תחת השם: sliding window

תת שלב מס' 4: הנחיתה בשרת (No Error & In-Order Delivery)

- שני הסגמנטים מגיעים לכרטיס הרשת של שרת ה-Apache (נניח שהגיעו בסדר הנכון וללא פגע).
 - פסיקת חומרה (SoftIRQ), שימקורה ב-NIC של מחשב השרת, מקפיצה אותם אל הקרנל של השרת.
 - בדיקת No Error:** הקרנל במחשב השרת, TCP 'הנמען', מחשב מחדש את ה-Checksum של כל סגמנט.
 - אם** התוצאה תואמת למה שכתוב בכותרת? מצוין. הסגמנט הגיע תקין $J = \text{לא}$ ללא שיבוש אם ביט אחד היה מתהפך, הסגמנט היה נזרק לפח באותו הרגע.
 - בדיקת רצף In-Order:** הקרנל בודק את מספרי הרצף. סגמנט 1 הגיע עם 1,000,001. הקרנל ציפה לזה! סגמנט 2 הגיע עם 1,001,461. הוא בדיוק ממשיך את הקודם. הרצף מושלם.
- הקרנל משווה את הדרך בשדה ה-Sequence Number עם ערכו של RCV.NXT שלו.
- קידום מצביע: ה- RCV.NXT מקודם ל-1,002,001 (הבית הבא המצופה).
- אחסון: הקרנל "מקלף" את כותרות ה-TCP מהסגמנטים, ומעתיק את 2000 הבתים של המידע, ומאחסן אותם בתוך **Buffer הקבלה** (sk_rcvbuf) של השרת, שהוא כאמור בגודל 128 KB. כעת פנויים שם רק 126 KB.

טיימרים (The Timing Mechanism)

ברגע שהסגמנט הראשון יוצא פיזית מה-NIC, הקרנל מפעיל את **טיימר ה-RTO (Retransmission Timeout)**. הוא רושם את חותמת הזמן הנוכחית. אם יגיע ה-ACK בתוך הזמן המוגדר – הטיימר יכובה. אם לא – ה-TCP יבצע שידור חוזר.

תת שלב מס' 5: האישור המצטבר (Cumulative ACK)

- כעת מגיע ה-TCP של השרת חייב להודיע ללקוח שהכל עבר בשלום [= הסגמנטים הגיעו אליו תקינים]
- לפי האפיון של פרוטוקול ה-TCP, 'המקבלי מחויב לאשר לישולח' את ההגעה של סגמנטים
- הוא מייצר סגמנט בקרה חדש (DATA-ACK segment).
סגמנט זה לא כולל data. היינו: $Payload = 0$.
במקרה שלנו, השרת עדיין לא עיבד את ה-GET, ולכן אין לו נתונים משלו לשלוח חזרה (אין כאן הזדמנות ל-Piggybacking אמיתי שבו ה-ACK "רוכב" על נתוני HTTP Response, אלא זהו סגמנט ACK טהור).
 - Acknowledgment Number נקבע ל-1,002,001
 - זהו ערך ה-RCV.NXT
 - משמעות: "קיבלתי הכל ברצף עד 1,002,000, כעת אני מצפה ל-1,002,001."
 - ערכו של ה-ACK Flag נקבע ל-1: ללא דגל זה, TCP במחשב הלקוח יתעלם מהערך שבשדה Acknowledgment Number ויחשיב אותו כ"זבל".

הדגשה:

- **מנגנון ה-TCP: Cumulative ACK** לא מאשר "חבילות", הוא מאשר "רצף של בתים". מכיוון שהתקבלו ברצף מושלם הבתים מ-1,000,001 ועד 1,002,000 (סך הכל 2000 בתים), הקרנל מציב בשדה ה-Acknowledgment Number את הערך **1,002,001**.
- **המסר מאחורי המספר**: "קיבלתי את כל הבתים עד לבית 1,002,000, ואני מצפה שהבית הבא שתשלח לי יתחיל באינדקס 1,002,001".
- הערה: אם סגמנט 1 היה הולך לאיבוד ורק סגמנט 2 היה מגיע, השרת היה משדר ACK עם המספר 1,000,001, ומאחסן את סגמנט 2 מחוץ לסדר, מה שהיה מפעיל את ה-Selective ACK – SACK, אליו נגיע בתסריט ב'.

תת שלב מס' 6: 'הצרכן' [= תוכנית ה-Apache] מתעורר והמעגל נסגר

- בזמן שחבילת ה-ACK עושה את דרכה חזרה, מתרחשים שני דברים במקביל:
1. **בצד השרת (Apache "הצרכן")**: הקרנל של השרת שולח אות התעוררות לתהליכון ה-Apache שישן.

Apache מבצע קריאת `recv(fd, buffer, ...)` ומבקש נתונים.

ה-2000 בתים מועתקים מתוך sk_rcvbuf של הקרנל אל הזיכרון הפרטי של Apache, ו- sk_rcvbuf מתרוקן בחזרה (מה שמגדיל מחדש את חלון הקבלה שיפורסם בעתיד).

Apache מתחיל לנתח את בקשת ה-GET ולחפש את קובץ ה-HTML.

2. **בצד הלקוח (ניקוי ו-Estimated RTT):** חבילת ה-ACK הגיעה אל הקרנל של Chrome.
 - הקרנל מסתכל על ה- Acknowledgment Number (1,002,001).
 - הוא מבין ששני הסגמנטים (שכאמור, הוחזקו כ"בני ערובה" ב- sk_sndbuf) הגיעו ליעדם ב בטחה.
 - רק כעת, הוא מוחק אותם לחלוטין מ-Buffer השליחה, ומשחרר את הזיכרון.
 - **חישוב Estimated RTT:** הקרנל בודק את השעון (Timestamps). הוא מחשב בדיוק כמה מילי-שניות עברו מרגע שידור סגמנט 1 ועד לקבלת ה-ACK הזה. מדידה זו (Sample RTT) מוזנת לנוסחת ההחלקה (Smoothed RTT), והקרנל מעדכן את הטיימר לעתיד למקרה שיצטרך לבצע Retransmission.

תת שלב מס' 7: התאמה דינמית וחישובי RTT (הדינמיקה)

כשה-ACK חוזר ל- TCP במחשב הלקוח, הקרנל מבצע פעולות אופטימיזציה קריטיות:

1. **ניקוי:** הסגמנטים נמחקים מ-sk_sndbuf.
2. **מדידת RTT:** הקרנל מחשב את ה-SampleRTT (זמן ההלוך-חזור).
3. **חישוב ה-Smoothed RTT:** הוא מעדכן את ה-EstimatedRTT באמצעות ממוצע נע:

$$EstimatedRTT = (1 - 0.125) \cdot EstimatedRTT + 0.125 \cdot SampleRTT$$

4 חישוב ה-Dev RTT (סטיית התקן):

$$DevRTT = (1 - 0.25) \cdot DevRTT + 0.25 \cdot |SampleRTT - EstimatedRTT|$$

5 עדכון ה-RTO:

$$RTO = EstimatedRTT + 4 \cdot DevRTT$$

הקרנל מסיים את הפעולה כשהוא מחזיק שעון עצר מכויל ומדויק להפליא עבור הסגמנטים הבאים.

סוף תסריט א'.

הלקוח [= http client שלח הודעת http request מסוג GET, הקיטוע עבד, מנגנוני הבקרה אישרו מעבר מידי, http server קלט ואישר.

וכעת תכנית ה- Chrome ממתינה לתגובת ה-HTTP של שרת ה-Apache.

לסיום:

התיאור הזה מאחד את האפליקציה, את ניהול הזיכרון בקרנל, את האריתמטיקה של מספרי הרצף, ואת מנגנוני הדיוק של הטיימרים.